Comparative Analysis of Centralized and Distributed Tree Indexing Techniques for Big Data

Da Zhang
Dept. of Engineering
University of Victoria
dazhang@uvic.ca

Yuanming Wang
Dept. of Engineering
University of Victoria
yuanmingwang@uvic.ca

***Abstract*—This paper presents a comparative study of centralized and distributed multi-dimensional tree indexing techniques for big data workloads. We implement B+ Tree, R-Tree, KD-Tree, and Quad-Tree in two settings: a local Python environment and a distributed PySpark environment that emulates multi-node execution using parallel partitions. Using a dataset of 2,000,000 synthetic key-value records, we evaluate index construction time, point-search latency, and range-query performance. We further analyze the impact of tree order on runtime and structural efficiency in both architectures. The study quantifies the trade-off between distributed-system overhead and parallel speedup, and identifies workload scenarios in which centralized or distributed indexing is more effective. Our results provide practical guidance for selecting tree-based indexing strategies in large-scale data management systems. Our implementation code is available at https://github.com/yuanmingwang/massive-dataset-indexing.**

***Index Terms*—big data indexing, centralized indexing, distributed indexing, B+ Tree, R-Tree, KD-Tree, Quad-Tree, PySpark**

I. Introduction

Traditionally, data management relied on centralized computing environments utilizing structures like the B+Tree for efficient one-dimensional indexing. However, the sheer volume of Big Data necessitates a paradigm shift towards distributed computing systems, which leverage frameworks such as MapReduce and Spark to partition data and execute operations in parallel. While numerous tree-based structures—including R-Tree, KD-Tree, and Quad-Tree—have been developed to handle multi-dimensional data, a direct, empirical comparison of their performance across both centralized and distributed environments remains a key research gap. Specifically, there is limited comprehensive data on how the inherent overhead of distributed systems compares to the substantial speed gains achieved through parallelized index construction and query execution.

This paper addresses this gap by presenting a rigorous comparative performance evaluation of these four core indexing structures. We implement and test the B+Tree, R-Tree, KD-Tree, and Quad-Tree in two distinct settings: a centralized environment using standard Python and a simulated distributed environment utilizing PySpark to mimic a Distributed File System (DFS). Our analysis focuses on the efficiency of these structures for two fundamental operations: point searching and range queries, using a dataset of 2,000,000 randomly generated key-value pairs.

II. Tree Structure Comparison

A. B+ Tree

The B+Tree is a fundamental self-balancing tree data structure primarily used for one-dimensional data indexing within database management systems, ensuring efficient insertions, deletions, and searches. Structurally, it is characterized by two main node types. Internal (non-leaf) nodes function purely as an index, storing keys and pointers to the next-level nodes to guide navigation. Crucially, all data pointers (records) are stored exclusively in the leaf nodes. These leaf nodes are interconnected sequentially to form a linked list, a feature that significantly facilitates efficient range queries. A key design principle is that all leaves reside at the same level, which guarantees a balanced height and therefore a predictable search time. By keeping data records out of the internal nodes, the B+Tree maximizes its branching factor, minimizing the tree's overall height and reducing the necessary disk I/O operations. It is specifically designed for 1D (single-key) data.

B. R-Tree

The R-Tree is a hierarchical, multi-dimensional extension of the B-Tree concept, designed specifically for spatial access methods to index spatial objects such as points, lines, or polygons. Each node in the R-Tree corresponds to a collection of entries. Every entry within a node contains a pointer to a child node and a minimum bounding rectangle (MBR). The MBR is defined as the smallest rectangle that entirely encloses all the objects or all the MBRs contained within its child node. A defining characteristic of the R-Tree is that the bounding rectangles of different nodes are allowed to overlap, which means a single search operation might need to traverse multiple paths of the tree to ensure the desired data is located. The core objective during construction and maintenance is to minimize both the overlap and the total area of the MBRs to maximize query performance. However, the logic for insertion and deletion is inherently complex due to the necessity of dynamically maintaining these MBR properties and handling potential node splits. The R-Tree is used for multi-dimensional spatial data (2D and higher).

C. KD-Tree

The KD-Tree is a space-partitioning data structure that organizes points within a k-dimensional space, proving particularly

effective for operations like nearest neighbor searches and range queries. It operates as a binary search tree, but each level of the tree uses a different dimension—the discriminating dimension—to partition the data space. For example, the tree might alternate between the X, Y, Z, and subsequent coordinates. A node stores a point and defines the split based on the value in the current discriminating dimension: all points in the left subtree have a lower value, and those in the right subtree have a greater value. This splitting process recursively partitions the space into hyper-rectangles (e.g., rectangles in 2D). The KD-Tree is well-suited for low to moderate dimensions, though its performance can decline in very high-dimensional spaces, a phenomenon known as the "curse of dimensionality." While the tree can be built and balanced to be static for optimal query speed, insertions and deletions are complex and often require rebuilding significant sections of the structure. It is designed for multi-dimensional point data.

D. Quad-Tree

The Quad-Tree is a hierarchical tree data structure used for the recursive partitioning of a two-dimensional space into four distinct quadrants or regions. Each internal node in a Quad-Tree is structured to have exactly four children, corresponding to the four regions—North-West, North-East, South-West, and South-East—created by bisecting the area defined by the parent node. The process of subdivision continues until a specific criterion is met, such as the region containing fewer than a certain number of points or becoming spatially homogeneous. This structure is widely utilized for spatial partitioning in applications like computer graphics, image analysis, and geographical indexing. Given this project's focus, it is likely using a Point Quad-Tree variant for indexing key-value pairs. Its construction and visualization of spatial partitioning are relatively straightforward, but similar to the KD-Tree, its height and balance are dependent on the spatial distribution of the underlying data. It is primarily used for 2D spatial data.

III. Point Searching & Range Query of Trees

A. B+ Tree

The B+Tree is optimized for one-dimensional data and sequential access, making both operations highly efficient.

i) Point Searching: For search of key K, the process begins by traversing internal nodes from the root. At each internal node, key K determines which child pointer to follow by comparing K with separator keys in that node. Traversal continues until a leaf node is reached. At the leaf node, entries are scanned for key K. If K is found, the associated data pointer is followed to retrieve the full record; otherwise, the key is confirmed to be absent from the tree.

ii) Range Query: For searching keys in range $[K_{\min}, K_{\max}]$, the first step is to run a point search for lower bound $K_{\min}$ to locate the starting leaf node. From that position, the query performs a sequential scan and outputs all records with keys in $[K_{\min}, K_{\max}]$. The scan continues by following next-pointers to subsequent leaf nodes in the linked list, and it terminates when a key greater than $K_{\max}$ is encountered.

B. R-Tree

The R-Tree is designed for multi-dimensional spatial data, using minimum bounding rectangles (MBRs) to index its nodes.

i) Point Searching: For search of point P, the process starts at the root node. In each internal node, every MBR entry is checked to determine whether it spatially contains target point P. Because MBRs can overlap, the search may need to follow multiple branches (child pointers) that enclose P. This check-and-traverse process is repeated recursively until a leaf node is reached. Within the leaf node, a final check is performed on the actual spatial object entries (or their MBRs) to precisely locate the object that matches point P.

ii) Range Query: For search of region R, traversal again starts at the root. At each internal node, the query checks which MBR entries intersect query region R. It then recursively follows all child pointers whose MBRs intersect R, since relevant spatial objects may exist in any intersecting branch. Upon reaching a leaf node, the contained spatial objects become candidate results. A final exact filtering step is performed on each candidate object to confirm it truly intersects with, or is contained within, query region R before being added to the result set.

C. KD-Tree

The KD-Tree partitions multi-dimensional point data by alternating the dimension used for splitting at each level.

i) Point Searching: For search of point P, the search begins at the root. At each current node, the coordinate of target point P is compared with the coordinate of the point stored at that node, using the node's current discriminating dimension (e.g., X, then Y, then Z). The search path is determined by this comparison: if P's coordinate is less than the node's coordinate, traversal moves to the left child; if it is greater than or equal, traversal moves to the right child. This comparison-and-navigation process repeats until the current node matches point P, or a leaf is reached without a match.

ii) Range Query: For search of region R (a hyper-rectangle), traversal starts at the root while implicitly tracking the spatial region represented by each node. There are three main cases. First, if the current node's region is entirely outside query region R, the entire subtree is pruned and search stops for that branch. Second, if the current node's region is entirely contained within R, all points in that subtree are added to the result set without further checking. Third, if the current region intersects R but is not fully contained, the point at the current node is tested and included if it lies inside R, then search recursively continues on both left and right child subtrees.

D. Quad-Tree

The Quad-Tree is a hierarchical structure that recursively partitions a 2D space into four quadrants.

i) Point Searching: For search of point P, the process begins at the root node, which represents the entire space. At each

current node, coordinates of target point P are compared with the node's spatial split (center point) to determine exactly which of the four quadrants (NW, NE, SW, SE) contains P. The search then traverses to the corresponding child node for that quadrant. This process repeats until a leaf node is reached. In a point Quad-Tree, the leaf either contains sought point P, concluding the search successfully, or is empty, indicating the point is not in the tree.

ii) Range Query: For search of region R (a 2D rectangle), traversal also starts at the root. The key step is identifying which of the four quadrants defined at the current node spatially intersect query region R. The algorithm recursively traverses only those child nodes (quadrants) that intersect R. If a current node is a leaf containing a point, that point is tested against region R and added to the result set if it lies inside. Traversal completes when all intersecting branches have been fully explored.

IV. Centralized Indexing Construction and Searching

i) Data Generation

Depending on the index type, different objects are generated:

`B+Tree`: random (key, record_id) pairs.

`RTree`: random rectangles in a 2D space.

`KDTree` / `QuadTree`: random 2D points.

These objects are created using a random generator with a fixed seed for reproducibility.

ii) Index/Tree Construction

Each local structure is built entirely in memory. `BPlusTree` is built by repeated insertions with node splits. `RTree` is built by repeated rectangle insertion with subtree selection and MBR updates. `KDTree` is built recursively by alternating split axes and choosing median points. `QuadTree` starts from one bounding box and subdivides into four children when a bucket overflows.

iii) Point Searching & Range Query

All local trees prune the search space with their structure. `BPlusTree` follows separator keys to one leaf for point search and scans linked leaves for ranges. `RTree` follows only MBRs that contain or intersect the query. `KDTree` prunes by axis-aligned split planes. `QuadTree` prunes by quadrant bounds and descends only into relevant cells.

V. Distributed Indexing Construction and Searching using MapReduce in PySpark

i) Data Generation and Distribution

Distributed demos use the same synthetic generators as the local versions, then assign each record to a Spark partition during the map stage. `BPlusTree` partitions by key range. `RTree` and `KDTree` use spatial grid-style partitioning over global bounds. `QuadTree` uses recursive quadrant-based partition IDs.

ii) Index/Tree Construction using MapReduce

After partitioning, each reducer partition rebuilds one local tree from its assigned records. That partition also emits small metadata such as bounds, item count, and height. The driver collects this metadata and keeps a lightweight routing layer over partitions.

iii) Point Searching & Range Query using MapReduce

Queries are first routed with the collected partition metadata so only candidate partitions are searched. Point queries usually touch one or a few partitions; range queries touch all overlapping partitions. Each selected partition rebuilds or reuses its local tree logic, and Spark collects the partial answers into the final result.

VI. Implementation Explanation

i) Experimental Environment and Tools

All implementations are developed in Python, with two execution environments: a standalone centralized runtime and a distributed PySpark runtime. The centralized implementation is executed on a single machine for direct in-memory indexing, while the distributed implementation uses Spark partitions to emulate multi-node processing under a MapReduce-style workflow. Runtime measurement is performed with consistent timing procedures across both environments to ensure fair comparison.

ii) Centralized Implementation Workflow

In the centralized workflow, data records are generated and inserted into each tree structure (B+ Tree, R-Tree, KD-Tree, and Quad-Tree) within one process. Index construction proceeds record by record according to each tree's split and balancing rules. Point search and range query operations are then executed directly against the constructed in-memory index. For each operation type, execution time is collected repeatedly and summarized to characterize average performance and variance under different tree parameters.

iii) Distributed Implementation Workflow in PySpark

In the distributed workflow, the same dataset is partitioned into multiple RDD/DataFrame partitions and processed in parallel. Mapper-side tasks construct local index fragments for their partitions, while reducer-side aggregation combines metadata or partial results to form a global query view. Point searches dispatch target keys to relevant partitions, and range queries evaluate candidate partitions concurrently before merging outputs. This implementation captures both parallel speedup and distributed overhead, including task scheduling, shuffle, and result consolidation costs.

iv) Reproducibility and Parameter Control

To keep results reproducible, dataset size, random seed, tree order/capacity, partition count, and query workload composition are fixed for each experiment group. Parameter sweeps (for example, varying B+ Tree order or Spark partition number) are run independently while holding other factors constant. This controlled implementation strategy isolates the influence of each design variable on construction time and query latency.

VII. Example of Distributed Implementation Workflow - R-Tree

Figure 1 provides an end-to-end view of the distributed `RTree` workflow used in this study. It summarizes how spatial

records are partitioned, how each reducer builds a local tree and emits routing metadata, and how query execution is routed, searched in parallel, and merged into final results.

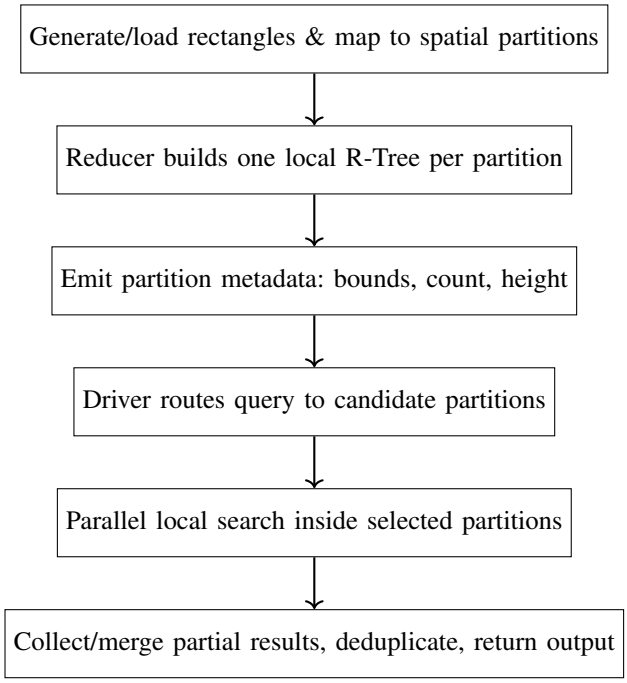

Fig. 1. Distributed R-Tree workflow in PySpark (MapReduce-style).

i) Mapper-Side Spatial Partitioning

The distributed R-Tree workflow starts by generating or loading spatial objects (rectangles) and assigning each object to a partition based on a global spatial grid. During the map stage, each record is tagged with a partition identifier derived from its location (or overlap) with grid cells, so mapper output is already grouped for local spatial indexing.

```
# Map build phase:
# map each rectangle to a spatial partition id, then
     partitionBy(...) shuffles
# rectangles so one reducer partition can build one
    local R-tree.
partitioned = (
    records_rdd.map(lambda row: (
        partition_id_for_rect(row[0]), row))
    .partitionBy(n_partitions, partitionFunc=lambda
        partition_id: partition_id)
)
```

ii) Reducer-Side Local R-Tree Build

Each reducer receives one partition's records and builds a local R-Tree in memory using standard rectangle insertion with subtree selection and MBR updates. After construction, the reducer emits compact metadata for routing, including partition MBR bounds, item count, and optional tree statistics (e.g., height or node count).

```
def prepare_partition(partition_id: int, iterator:
    Iterator[Tuple[int, Tuple[Rect, int]]]):
    # Reduce/build phase:
    # rebuild one local R-tree from this Spark
        partition and emit partition MBR
    # metadata so the driver can route later queries
        .
    rows = [row for _partition_key, row in iterator]
    local_tree = RTree(max_entries=max_entries)
    if rows:
        local_tree.build(rows)
        partition_mbr = Rect.enclosing(rect for rect
            , _ in rows)
        metadata = {
            "partition_id": partition_id,
            "n_items": len(rows),
            "mbr": partition_mbr,
            "height": local_tree.height(),
        }
    else:
        metadata = {
            "partition_id": partition_id,
            "n_items": 0,
            "mbr": None,
            "height": 0,
        }
    yield (partition_id, rows, metadata)

self.partition_data_rdd = partitioned.
    mapPartitionsWithIndex(prepare_partition)
```

iii) Query Routing and Parallel Search

For point queries and range queries, the driver first checks partition metadata to identify candidate partitions whose bounds may satisfy the query. Spark then dispatches the query only to those partitions. Within each selected partition, the local R-Tree performs normal MBR-based pruning and returns partial matches.

iv) Result Merge and Final Output

Partial results returned by candidate partitions are merged at the driver. For point queries, this typically involves few partitions; for range queries, multiple overlapping partitions may contribute results. A final deduplication/filter step is applied if needed, and the merged set is returned as the final query result.

```
def point_query(self, point: Point):
    candidate_partitions = set(
        int(bound["partition_id"])
        for bound in self.partition_bounds
        if bound["mbr"] is not None and bound["mbr"
            ].contains_point(point)
    )
    max_entries = self.max_entries

    def search_partitions(iterator: Iterator[Tuple[
        int, List[Tuple[Rect, int]], Dict[str,
        object]]]):
        # Distributed point-search phase:
        # after driver-side routing by partition MBR
            , only candidate partitions
        # rebuild/search their local R-trees for
            rectangles containing the point.
        for partition_id, rows, _metadata in
            iterator:
            if partition_id in candidate_partitions
                and rows:
                local_tree = RTree(max_entries=
                    max_entries)
                local_tree.build(rows)
                yield from local_tree.query(point)

    return self.partition_data_rdd.mapPartitions(
        search_partitions).collect()
```

```python
def intersection_query(self, query_rect: Rect):
    candidate_partitions = set(self.
        _candidate_partitions_for_query(query_rect))
    max_entries = self.max_entries

    def search_partitions(iterator: Iterator[Tuple[
        int, List[Tuple[Rect, int]], Dict[str,
        object]]]):
        # Distributed range/intersection phase:
        # only partitions whose partition MBR
            intersects the query rectangle rebuild
        # and search their local R-trees before
            Spark collects the partial results.
        for partition_id, rows, _metadata in
            iterator:
            if partition_id in candidate_partitions
                and rows:
                local_tree = RTree(max_entries=
                    max_entries)
                local_tree.build(rows)
                yield from local_tree.query(
                    query_rect)

    return self.partition_data_rdd.mapPartitions(
        search_partitions).collect()
```

VIII. Results Comparison

i) Construction Time Comparison

a) Centralized Trees

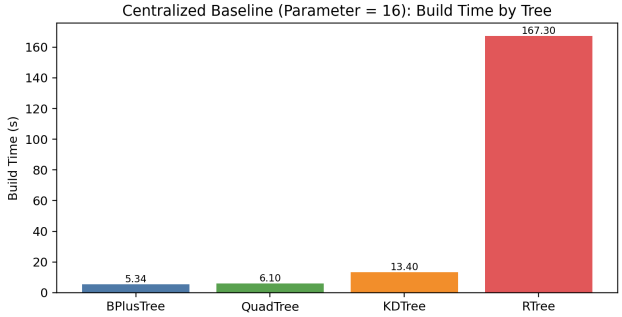

Fig. 2. Centralized baseline ($n = 2{,}000{,}000$, parameter = 16): build time by tree structure.

In this subsection, we use tree-order/capacity 16 as the baseline for centralized-time comparison across trees. Under this baseline at $n = 2{,}000{,}000$, `B+Tree` has the lowest construction time (about 5.34 s), followed by `QuadTree` (about 6.10 s), then `KDTree` (about 13.40 s), while `RTree` is by far the slowest (about 167.30 s). This ordering indicates that, under matched parameter setting 16, centralized build overhead is dominated by R-Tree insertion/MBR maintenance complexity, whereas B+Tree and QuadTree can be constructed much faster on the same data scale.

b) Distributed Trees

In this subsection, we compare distributed construction time using baseline parameter 16 (B+Tree order 16, KDTree leaf capacity 16, RTree max entries 16, and QuadTree depth baseline 16) at $n = 2{,}000{,}000$. `DistributedB+Tree` is fastest at about 3.84 s, followed by `DistributedKDTree` at about 8.11 s and `DistributedQuadTree` at about 11.79 s,

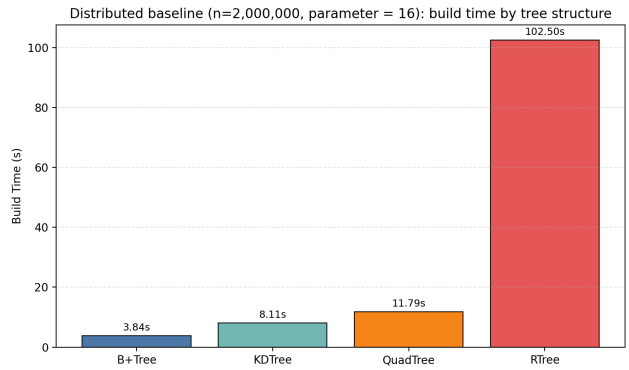

Fig. 3. Distributed baseline ($n = 2{,}000{,}000$, parameter = 16): build time by tree structure.

while `DistributedRTree` is much slower at about 102.50 s. The baseline ranking in distributed mode therefore remains B+Tree $<$ KDTree $<$ QuadTree $\ll$ RTree for build cost, indicating that R-Tree split/MBR-maintenance overhead is still the dominant factor even with parallel partitioned construction.

ii) Point and Range Query Performance

a) Centralized Trees

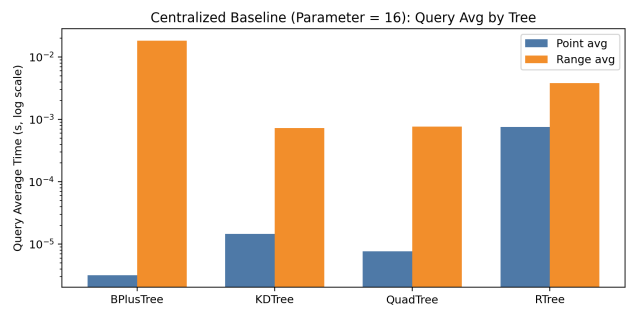

Fig. 4. Centralized baseline ($n = 2{,}000{,}000$, parameter = 16): point and range query average latency by tree structure (log scale).

The baseline centralized query behavior (parameter = 16) shows clear separation between point and range costs. Point-query latency remains very low for `B+Tree`, `KDTree`, and `QuadTree`, while range queries are consistently higher due to larger candidate/result traversal, with `RTree` showing the largest query-side overhead under overlap-heavy spatial search.

b) Distributed Trees

Using baseline parameter 16 for distributed query comparison, `DistributedB+Tree` has the lowest average latency (point $\approx$ 0.50 s, range $\approx$ 0.51 s), followed by `DistributedKDTree` (point ≈ 1.68 s, range ≈ 1.73 s) and `DistributedQuadTree` (point ≈ 4.16 s, range ≈ 4.16 s). `DistributedRTree` is highest in this baseline result (point ≈ 61.32 s, range/intersection ≈ 50.00 s), showing much larger query overhead under overlap-heavy spatial traversal and per-partition tree rebuild/search cost. Similar to the centralized query baseline, range/intersection workloads are generally

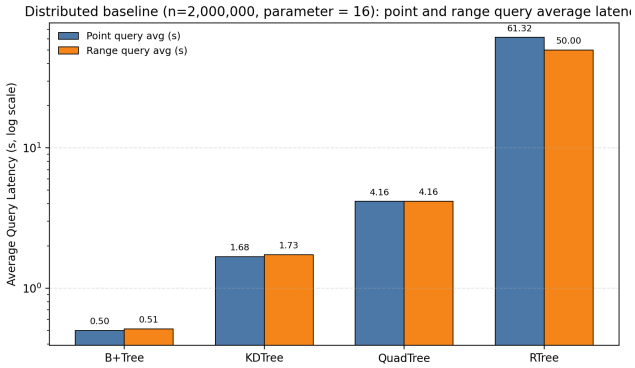

Fig. 5. Distributed baseline (n = 2,000,000, parameter = 16): point and range query average latency by tree structure (log scale).

not cheaper than point workloads for B+Tree, KDTree, and QuadTree due to broader candidate traversal.

iii) Comparison of Centralized Tree Orders on Query Performance

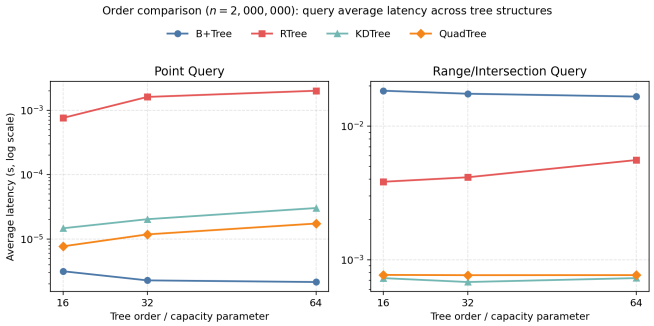

Fig. 6. Comparison of tree-order/capacity settings (16, 32, 64) on point and range/intersection query average latency at $n = 2{,}000{,}000$ (log scale).

Figure 6 compares query performance when the order/capacity parameter increases from 16 to 64. For `B+Tree`, both point and range-query latency decrease modestly as order grows, consistent with reduced tree height and fewer node visits. `KDTree` and `QuadTree` show very small changes for range workloads, but their point-query latency increases as capacity grows, indicating a trade-off where larger leaves reduce depth while increasing per-node scan cost. `RTree` shows the strongest degradation with larger max entries for both point and intersection queries, suggesting that larger node capacity increases overlap/candidate checks in this workload. Overall, for query-centric settings in this experiment, parameter 16 remains the best or near-best choice for `KDTree`, `QuadTree`, and `RTree`, while `B+Tree` benefits from larger order values.

IX. Conclusion

Our experimental results demonstrate that `B+Tree` consistently achieves the best overall performance, particularly in construction time and query latency, due to its balanced structure and efficient node traversal. In contrast, `RTree` exhibits significantly higher computational overhead in both centralized and distributed settings as a result of complex minimum bounding rectangle (MBR) maintenance and overlapping region traversal. `KDTree` and `QuadTree` provide a balanced trade-off between performance and flexibility for multi-dimensional data, though their efficiency depends on data distribution and dimensionality.

Furthermore, our study highlights an important trade-off between centralized and distributed systems. While distributed implementations using PySpark can significantly reduce construction time through parallelism, they also introduce non-negligible overhead from data partitioning, communication, and task coordination. As a result, distributed indexing is most beneficial for large-scale workloads, whereas centralized approaches remain competitive for moderate-sized datasets.

From a practical perspective, each tree structure aligns with different real-world applications. `B+Tree` is widely used in database systems and file systems, where efficient disk-based indexing and range queries are critical (e.g., MySQL, Oracle). `RTree` is commonly applied in spatial databases and geographic information systems (GIS), supporting location-based queries such as map search, route planning, and spatial joins. `KDTree` is frequently used in machine learning and computational geometry, particularly for nearest-neighbor search in tasks such as recommendation systems, image retrieval, and clustering. `QuadTree` is well-suited for hierarchical spatial partitioning, making it useful in computer graphics, image compression, and game development for collision detection and rendering optimization.

X. References

[1] Samoladas, Dimitrios; Karras, Christos; Karras, Aristeidis; Theodorakopoulos, Leonidas; and Sioutas, Spyros (2023). “Tree Data Structures and Efficient Indexing Techniques for Big Data Management: A Comprehensive Study,” pp. 123–132. DOI: 10.1145/3575879.3575977.